Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.0.2 sur 153 | difficulté 0 / 10

0.0.2. La fonction texte() — écrire un mot

#include <texte>, puis texte(colonne, ligne, "MOT") : le mot s'écrit à cette place, une lettre par case.

1. Ce qu'il faut comprendre

Une fonction, une étape. Avant d'employer une fonction de la console pour la première fois, le parcours la présente seule : la ligne `#include` qui la rend disponible, ses arguments, ce qu'elle coûte, et un essai. Ici, `texte()`, la première de toutes.

Ce qu'elle fait : `texte(colonne, ligne, "MOT")` écrit des lettres à l'écran, une par case, de gauche à droite. L'écran fait **20 colonnes** (0 à 19) et **18 lignes** (0 à 17).

La ligne à écrire : `#include <texte>`**.** Sans elle, le compilateur refuse `texte()` et dit quelle ligne écrire.

Ses arguments : la **colonne** de la première lettre, la **ligne**, puis le **texte entre guillemets** : des lettres, des chiffres, des espaces. La colonne et la ligne peuvent être des variables.

Ce qu'elle coûte : environ **150 octets** la première fois (la mise en place de l'écran, la routine qui écrit, les lettres employées), puis une dizaine d'octets par appel.

Essaie : écris ton prénom en (0, 17), tout en bas à gauche.

On retrouve texte dans 312 leçons

0.0.1, 0.1, 0.1.1, 0.1.2, 0.3, 0.3.1, 0.3.2, 0.4, 0.4.1, 0.4.2, 0.79, 0.79.1, 0.81, 0.81.1, 0.81.3, 0.82.1, 0.83, 0.83.2, 0.84, 0.87.7, 0.87.9, 0.88, 0.88.1, 0.88.2, 0.89.2, 0.90, 0.90.1, 0.90.2, 0.90.4, 0.90.5, 0.90.6, 0.90.7, 0.90.8, 0.91, 0.91.1, 0.91.2, 0.91.3, 0.91.4, 0.91.5, 0.91.6, 0.91.7, 0.91.8, 0.91.9, 0.92, 0.92.1, 0.92.2, 0.93.3, 0.93.4, 0.93.5, 0.93.6, 0.93.7, 0.93.8, 0.93.9, 0.94, 0.94.1, 0.95, 0.96, 0.96.1, 0.97, 0.97.1, 0.98, 0.98.1, 0.99, 0.99.1, 0.100, 0.100.1, 0.101, 0.101.1, 0.102, 0.102.1, 0.103, 0.104, 0.105, 0.105.1, 0.106, 0.107, 0.107.1, 0.108, 0.1, 1, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.8, 1.9, 1.10, 1.11, 1.12, 2, 2.1, 2.2, 2.3, 4, 5, 6, 7, 8, 8.1, 9, 10, 11, 12, 12.1, 13, 14, 16, 17, 18, 19, 20, 20.1, 21, 22, 23, 24, 24.1, 25, 26, 27, 28, 29, 30, 31, 32, 33, 33.1, 34, 35, 35.12, 35.16, 35.19, 36, 37, 38, 39, 41, 43, 43.1, 44, 45, 46, 47, 52.1, 53, 54, 55, 56, 57, 58, 59, 59.1, 60, 65, 66, 66.1, 67, 68, 69, 70, 71, 71.1, 72, 73, 75, 77, 78, 78.1, 79, 80, 81, 82, 82.1, 83, 84, 85, 86, 90, 90.1, 91, 92, 93, 94, 94.1, 95, 96, 97, 98, 99, 100, 100.1, 101, 102, 103, 104, 105, 106, 106.1, 107, 108, 109, 110, 110.3, 110.6, 110.7, 110.9, 110.12, 110.13, 110.14, 110.15, 110.16, 110.18, 110.20, 110.21, 110.22, 111, 112, 113, 113.2, 113.5, 114, 115, 116, 117, 117.1, 118, 119, 120, 120.1, 121, 122, 123, 123.1, 124, 125.1, 126, 127, 128, 129, 133, 134, 134.1, 135, 136, 138, 139, 141, 143, 143.1, 152.1, 153.9, 2.01, 2.02, 2.03, 2.04, 2.05, 2.06, 2.07, 2.08, 2.09, 2.10, 2.11, 2.12, 2.13, 2.14, 2.15, 2.16, 2.17, 2.18, 2.19, 2.20, 2.21, 2.22, 2.23, 2.24, 2.25, 2.26, 2.27, 2.28, 2.29, 2.30, 2.31, 2.32, 2.33, 2.34, 2.35, 2.36, 2.37, 2.38, 2.39, 3.01, 3.02, 3.03, 3.04, 3.05, 3.06, 3.07, 3.08, 3.09, 3.10, 3.11, 3.12, 3.13, 3.14, 3.15, 3.16, 3.17, 3.18, 3.19, 3.20

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <texte> : écrire un mot ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <texte> : écrire un mot ---- »). La console l'ignore.
`// texte(colonne, ligne, "MOT") écrit le mot à cette place, une lettre par case.`	Un commentaire : une phrase pour celui qui lit (« texte(colonne, ligne, "MOT") écrit le mot à cette place, une lettre par case. »). La console l'ignore.
`#include <texte>  // texte() : sans cette ligne, le compilateur la refuse`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « texte() : sans cette ligne, le compilateur la refuse ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.

`texte(4, 8, "BONJOUR");  // colonne 4, ligne 8 : B en (4, 8), O en (5, 8)…`	Appel de la fonction texte de la console, avec les arguments 4, 8, "BONJOUR". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « colonne 4, ligne 8 : B en (4, 8), O en (5, 8)… ».)
`texte(0, 0, "EN HAUT");  // la case (0, 0) est en haut à gauche`	Appel de la fonction texte de la console, avec les arguments 0, 0, "EN HAUT". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « la case (0, 0) est en haut à gauche ».)
`while (true) {  // la boucle du jeu : rien d'autre à faire`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « la boucle du jeu : rien d'autre à faire ».)
`image();  // image() est native : elle ne s'inclut pas`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « image() est native : elle ne s'inclut pas ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **425 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           29 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     85 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          160 o  ← CopierTuiles  — 10 lettres sur 43
  "BONJOUR"         7 o  ← texte()  — un texte gravé
  "EN HAUT"         7 o  ← texte()  — un texte gravé
Lettres gravées : ABEHJNORTU
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 2    22 o sur place, dans main()  + EcrireTexte (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <texte> : écrire un mot ----
// texte(colonne, ligne, "MOT") écrit le mot à cette place, une lettre par case.

#include <texte>   // texte() : sans cette ligne, le compilateur la refuse
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

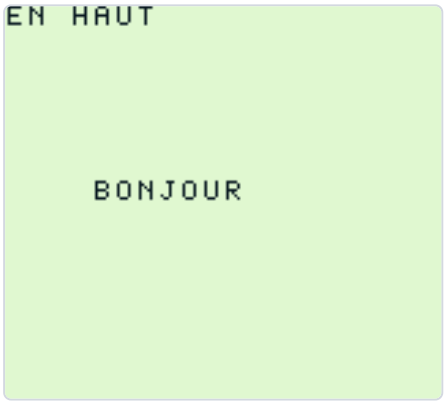
```
int main() {
  texte(4, 8, "BONJOUR");   // colonne 4, ligne 8 : B en (4, 8), O en (5, 8)…
  texte(0, 0, "EN HAUT");   // la case (0, 0) est en haut à gauche

  while (true) {            // la boucle du jeu : rien d'autre à faire
    image();                // image() est native : elle ne s'inclut pas
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> BONJOUR au milieu de l'écran, EN HAUT dans le coin en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ BONJOUR est en (4, 8) (« BONJOUR »)

✓ EN HAUT est en (0, 0)